Real-Time Sign Language (RTSL)

Version 0

Group 23

COMPSCI 4ZP6A

Arin Khandelwal	khanda3@mcmaster.ca
Dhruv Chand	chandd9@mcmaster.ca
Gaurav Sharma	sharmg36@mcmaster.ca
Ha Nguyen	nguyeh83@mcmaster.ca
Malko-Bani Somo	somom1@mcmaster.ca
Roshaan Quyum	quyumr@mcmaster.ca
Vinoth Robinson	robinv4@mcmaster.ca

Table of Contents

0. Document Information 4
0.1. Abbreviations, Notation, Naming Conventions, and Definitions 4
0.2. Contribution History 4
0.3. Revision History 4
1. Purpose: 5
1.1 Background 5
1.2 Project Objectives 5
1.3 Proposed Solution 5
2. The Client and Stakeholders: 6
3. Project Constraints: 6
3.1 Hardware/Platform Constraints 6
3.2 Performance Constraints 6
3.3 Testing Constraints: 6
3.4 Environment Constraints: 6
3.5 Data Constraints: 6
3.6 Schedule Constraints: 7
4. Functional Requirements: 7
4.1 Priority 0 (Minimum Viable Product): 7
4.2 Priority 1 (Important but Non-Critical Features): 7
4.3 Priority 2 (Useful but Non-Critical Features): 7
4.4 Priority 3 (High Cost, Low Impact Features): 8
4.5 Priority 4 (Future Application Features): 8
5. Data and Metrics: 8
5.1 Dataset Features 8
5.2 Links to Datasets 9
5.3 Performance Metrics and Expectations 10
6. Non-Functional Requirements: 10
6.1 Look and Feel 10
6.2 Usability and Humanity 11
6.3 Performance and Speed 11
6.4 Security and Privacy 11
6.5 Legal 11
7. Risks and Issues Predicted: 12
7.1 Technical Risks/Issues 12
7.2 Deployment Risks/Issues 12
7.3 Limited sentence-level data 12
8. Team Meeting and Communication Plan: 12
8.1 Team Meetings 12
8.2 Communication Plans 13
9. Team Member Roles: 13
10. Workflow Plan: 14
10.1 Version Control 14
10.2 Development Methodology 15

10.3 Data Storage....16
10.4 Compute-heavy Tasks....16
10.5 Accomplishing Requirements....16
11. Proof of Concept Demonstration Plan:....16
12. Technology:....18
12.1 Front-end....18
12.2 Back-end....18
12.3 Code Environment:....19
13. Project Scheduling:....19

0. Document Information

0.1. Abbreviations, Notation, Naming Conventions, and Definitions

RTSL	This stands for Real-Time Sign Language, which is the name of our project
ASL	This stands for American Sign Language, communicated through gestures.
WLASL	This stands for Word-Level American Sign Language. It is a video dataset containing approximately 2000 words performed by over 100 signers.
ML	This stands for Machine Learning.
PR	This stands for Pull Request.
PWA	This stands for Progressive Web App
GPU	This stands for Graphics Processing Unit
ROC Curve	This stands for Receiver Operating Characteristic Curve, a graph detailing the performance of a binary classification model at various thresholds.

0.2. Contribution History

Author	**Contributions**
Arin Khandelwal	3, 7, 11
Dhruv Chand	2, 3, 5, 7
Gaurav Sharma	0, 4, 5.3, 9, 10
Ha Nguyen	2, 3, 7
Malko-Bani Somo	5, 6, 12
Roshaan Quyum	5, 6, 12
Vinoth Robinson	0, 1, 5, 8, 13

0.3. Revision History

Version	**Date**	**Changes**	**Authors**
0	October 10th, 2025	Initial Document	Arin Khandelwal, Dhruv Chand, Gaurav Sharma, Ha Nguyen, Malko-Bani Somo, Roshaan Quyum, Vinoth Robinson
1	December 6th 2025	Updated Functional Requirements	Arin Khandelwal, Dhruv Chand, Gaurav Sharma, Ha Nguyen, Malko-Bani Somo, Roshaan Quyum, Vinoth Robinson

1. Purpose:

1.1 Background

From the onset of the project, our team wanted to create a tool that would help people. In our research for topics and data, we found datasets related to ASL and decided that we wanted to try to create an application that can translate ASL to English. This would significantly aid in the communication between people who use ASL and those who do not.

Although there are applications that can translate ASL to English, in our search, we could not find an easy to access real-time translation app. Therefore the goal of this project is to develop an open-source real-time ASL to English translation application that prioritizes accessibility.

1.2 Project Objectives

The goal of this project is to:

- Develop an app that can translate a string of ASL gestures to English in real time.
- Ensure that the translation occurs in real time.
- Make the application accessible.
    - Hosted on the web.
    - Doesn't require an account.
    - Translation automatically occurs after a press of a button and consent has been given.
- If time permits, develop a web extension that can translate ASL to English during online meetings

1.3 Proposed Solution

The end goal of our project, Real-Time Sign Language (RTSL), is to be an accessible web application relying on machine learning to convert ASL gestures to written English, helping bridge the gap between ASL users and non-ASL users. To ensure ease of use and protect user privacy, the application will not require an account and no user data will be stored. As the project progresses, we initially hoped to include games and other fun activities that help teach and engage non-ASL users in learning sign language, but we are now hoping to create a web extension that will allow you to translate ASL during meetings.

2. The Client and Stakeholders:

Our tool primarily serves two client groups. The first group is people who don't understand ASL and need a tool to conveniently translate it for daily use. For example, families with a deaf member could use this tool to communicate at home more naturally. The second group consists of people who are actively learning or fluent in ASL. For instance, the ASL club at McMaster could use this tool to practice their signing and receive feedback instantly for self-study sessions.

Our team is currently discussing with the McMaster ASL club for software validation with members who are familiar with ASL. This ensures our system works well through real-world testing. Our tool is expected to address a significant communication gap while promoting accessibility and inclusion for the deaf community.

3. Project Constraints:

3.1 Hardware/Platform Constraints

- The application must be able to run smoothly on both mobile and computer platforms.
- Browser compatibility: Chrome, Safari, Firefox, Edge, etc.
- OS compatibility: iOS, Android, Windows, macOS.

3.2 Performance Constraints

- The model must be able to run efficiently enough where translation occurs in real time or near real time.
- Web application must be able to handle traffic.
- Be able to handle dropped frames or lost packets while streaming.

3.3 Testing Constraints:

- Must test against standardized ASL datasets.
- User acceptance testing by letting real ASL users validate real-world usability.
- Videos of people signing from different angles should be tested against.

3.4 Environment Constraints:

- Lighting condition/ background complexity/ camera positioning.

3.5 Data Constraints:

- Dataset diversity:
    - Variation in skin tones, hand shapes, signing speeds, genders, etc.
    - Vocabulary scope (videos of 2000 common words)
- Sentence-level data:
    - Turning words and facial expressions to sentences
    - Handle different signing speeds
- Privacy
    - Handling video streams by uploading to the cloud
    - Policies (do not start video recording without user's consent)

3.6 Schedule Constraints:

- There should be Application update checkpoints to keep track of progress.
- We need to define Phases for this project. To divide up the project progress in phases and assign a timeline to these phases.

4. Functional Requirements:

4.1 Priority 0 (Minimum Viable Product):

- A video-based ML model that can convert 2000 unique ASL gestures to English words in a live setting with approximately 70% accuracy.
    - Using datasets like 21,000 WLASL Videos, ASL Skeletal Sign Language Videos, and others outlined in section 5.2.
- A modern website that allows users to input ASL gestures through a web camera and receive translations in near real time.
    - Users will have a live camera feed to see what the model's seeing.
    - Translations will be of each individual sign and the final sentence once the user pauses signing.
- A Transformer to convert the video ML model's word outputs (array of unconnected words/glosses) into a sentence.
- Integrating backend to frontend connectivity using WebSockets.
    - Sends user input (keypoints of hands and body) to model.
    - Sends a predicted word or sentence to the application in response to input.
- Unit testing for backend and frontend.
    - Tests for building the application.
    - Tests for ensuring reliability of frontend components.
    - Tests for ensuring correctness of preprocessing components.
- Support for mobile device web browsers (Safari, Google Chrome, etc.).
    - Intuitive UI reliant on common mobile actions like tapping.

4.2 Priority 1 (Important but Non-Critical Features):

- Web extension allows screen capture as input instead of camera.
    - Enables use of application in video calls and other websites.
- Application's frontend and backend hosted on a server.
    - Application can be used without downloading locally.
    - Application is platform agnostic (compatible with different devices).
- Using text-to-speech to verbalize the sentence translation.
    - Toggled through a button, default is not on.
- Support for desktop/laptop web browsers.
    - Reactive scaling for different display resolutions.
- Real-time translation (under one second) with no noticeable delay.
- Outputting multiple interpretations of the sentence based on separate Transformer models.

4.3 Priority 2 (Useful but Non-Critical Features):

- Adding an “advanced” mode to the website to give more technical feedback to users.
    - Showing a bounding box of hands.
- A static, numbered scenario system that engages the user in responding to day-to-day situations in ASL (e.g. buying groceries from the store).
    - Has ten scenarios users can choose from.
- Using facial expressions to denote tone (e.g. happy or sad).
    - Using datasets that explicitly include faces in their videos like OpenASL: An Open-Domain Sign Language Translation Dataset and How2Sign Dataset
    - Also may use Grammatical Facial Expressions Dataset
- Updating text-to-speech to accurately convey tone with verbalization.

4.4 Priority 3 (High Cost, Low Impact Features):

- A dynamic scenario system that allows users to generate their own scenarios.
    - Having the scenarios themselves use ASL (3D rendering of arms).
    - Scenarios generated through LLM.
- Games to help engage users in sign language.
    - Translation questions (i.e, what is this sentence in ASL, like Duolingo).
    - Image responder (flash an image on screen, then sign it within a time limit, like GeoGuesser).

4.5 Priority 4 (Future Application Features):

- Expanding dataset to more than 2000 ASL gestures.
- Support for hundreds of devices simultaneously using the site.
- Employing differential privacy tactics on the dataset to protect data.
- Adding a report error feature allowing users to submit their own data on words commonly mislabeled to help train the model.
    - Would require storing user data in the database and dynamically updating the model.

5. Data and Metrics:

5.1 Dataset Features

Using the following features, we believe we will be able to accurately differentiate between all ASL gestures in our testing dataset.

- Length of the bounding box of the finger.
    - To determine the finger position, and in turn the sign being performed.
    - Raising and lowering fingers into different positions e.g clenching fingers to your thumb (👌) vs a fist (✊).

- Width of the bounding box of the palm.
    - To determine the angle at which the hand is positioned (palm facing inwards, hand on angle, etc.).
- Angle of each finger's bounding box to the palm.
    - To recognize signs from the hand when viewed from the side.
- Number of fingers touching the thumb.
    - To determine the sign the hand is showing.
- Hand bounding box overlapping with another bounding box.
    - To determine the type of gesture being produced by the person, e.g hand above mouth, stomach, or chest.
- Vertical and Horizontal movement of hand bounding box.
    - To determine the movement of the sign, as many signs include large amounts of hand movement.
- Velocity of bounded boxes
    - Keep track of pauses, in ASL words that consist of more than one gesture, there will be a pause (velocity of bounding boxes equal to 0) between the person signing the different gestures.
    - Gestures can be done at varying speeds.
    - Cycles, some gestures are repeated movements (going in a circle).

Depending on hand gestures, MediaPipe may not be able to draw bounding boxes around all parts of the hand / body because they may be obstructed.

Features are subject to change once we start working on the model and identify other features that we would need to track

5.2 Links to Datasets

We plan to use some of the following datasets when training our model. The ones that we use will be dependent on how useful they are when tracking the unique features of each ASL gesture. Additionally the splits we plan on using for training, validation and testing is 80%/10%/10% respectively.

- 21,000 WLASL Videos
- 12,000 WLASL Videos
- ASL Skeletal Sign Language Videos
- OpenASL: An Open-Domain Sign Language Translation Dataset
- How2Sign Dataset
- American Sign Language Linguistic Research Project Sign Bank
- ASL Alphabet Hand Skeleton Dataset
- ASL Alphanumeric Dataset
- Sign Language Recognition Dataset

- Grammatical Facial Expressions

5.3 Performance Metrics and Expectations

We will treat ASL translation as a multi-class classification problem where each sign is assigned a class corresponding to its English translation. This framing allows us to apply binary classification performance metrics on a per-word basis.

In this context, false negatives occur when the model fails to correctly identify the intended sign or fails to produce an output, and false positives occur when the model misclassifies a sign or produces an output when there is no sign. Importantly, each misclassification will simultaneously be a false negative for the intended sign and a false positive for the incorrectly predicted class. This dual impact makes both precision and recall critical for evaluation.

The first and most important metric we will use to evaluate our project is the model's recall measured on a word-by-word basis. Recall will measure the rate at which our model correctly identifies a sign and is defined as $= \frac{True\ Positive}{True\ Positive + False\ Negative}$. Although recall should be considered with FPR, we will prioritize maximizing recall, as failing to output the intended sign will result in a more significant breakdown in communication than mistakenly outputting an extra word. Given these considerations, our goal is to have a recall rate of approximately 80%.

The second metric we will use to evaluate our project is the model and algorithm's accuracy measured on a sentence-by-sentence basis. Accuracy will measure the rate at which our model correctly outputs the intended meaning of a sequence of gestures and is defined as $= \frac{Correct\ Predictions}{All\ Predictions}$. In this case, the intended meaning of a sequence of gestures will strictly be the label given to the data. Although accuracy is heavily affected by class imbalance, since each unique sentence can be considered its own class, we believe this metric will be unaffected and a clear indicator of how accurate our project is.

Lastly, the third metric will be used to evaluate the physical performance or efficiency of the overall project. This refers to the time it takes between signing and getting feedback from our system and accounts for the delay between sending data from user to model, delay processing data with model, and delay sending output of model back to user. We aim to have an end-to-end latency goal of <1s.

6. Non-Functional Requirements:

6.1 Look and Feel

The app will start with asking for user consent. Once consent is given, the front camera view will take up most of the whole screen, similar to Snapchat. The interface style will have a modern, minimalist look. Minimalism in particular will be emphasized to keep the app accessible.

6.2 Usability and Humanity

The interface should be intuitive and familiar so that the user is able to use the app without having to learn anything. There will be a menu which gives the user to select the camera translation functionality or the web extension. On the translation page, there will be a button at the bottom center of the screen which begins the translation when pressed. As the user signs, translations for each individual gesture will pop up. When the user pauses, a translation will be given for the entire sentence. The transition will also be stored in the chat history window.

6.3 Performance and Speed

To achieve "real-time translation", we will target an end-to-end latency goal of <1s, assuming we use a GPU server. This latency goal will depend on server architecture and the user's phone.

6.4 Security and Privacy

The app has no data persistence requirements. Any data sent to our server for translation will only be stored in-memory and will be discarded upon returning. Additionally, users will not be required to make an account to use the app. There will also be measures in place to detect if a user is attempting to overload the system (i.e they start the translation but do not sign).

6.5 Legal

6.5.1 Project License

The project is licensed under the GPLv3. We believe our work should be publicly available, to that end we are using datasets with permissive licenses and releasing our own work using the GPL.

6.5.2 Dataset Licenses

The majority of the datasets we use from kaggle are licensed under the Public Domain. Such datasets, and the 3 datasets listed below, can be freely used non-commercially.

- The Sign Language Recognition Dataset (MIT license).
- The Skeletal Sign Language Videos - ASL (MIT license).
- The How2Sign dataset (CC BY-NC 4.0 license).

The license of the WLASL dataset needs slightly more discussion. It is licensed under C-UDA-1.0. This license allows use of the dataset for solely "Computational Use". Training AI models are explicitly given as an example of what is allowed in the annotated version of the license (can be found here). Since we are following the terms of the license, we can use, modify, and redistribute the data.

7. Risks and Issues Predicted:

7.1 Technical Risks/Issues

- Potential signing misclassification due to similarities in handshape or movements.
    - Users may also try to not sign or simply do random hand motions which could get misclassified.
- Limited dataset may restrict vocab that the model will be able to translate.
- The model might not be fast enough to make translations seem to be occurring in real time and cause delays in translation.

7.2 Deployment Risks/Issues

- Devices with weak internet signals might cause problems such as delayed or incorrect translations.
- Devices with poor cameras might cause issues resulting in incorrect or misleading translations.
- Devices with low processing power might cause worse performance depending on how demanding the web application is.

7.3 Limited sentence-level data

- Having mostly isolated signs can lead to poor grammar/ordering in translation.
- More likely to result in "word-by-word" translation which lowers the quality of the sentences and communicating experience.
- Issues may arise if a single important is not translated properly (for example negations like "not").
    - On top of negations, words such as practically, virtually, barely, etc. can often change the meanings of sentences differently depending on what contexts they're found in.

8. Team Meeting and Communication Plan:

8.1 Team Meetings

Each team member filled out a "when2meet" to find the ideal time for meetings. Not including the time during lectures to meet, the team has decided to meet on Fridays every week, and if necessary, meet on Saturday and Sunday as well. The meetings will typically be virtual, and most of the work will be asynchronous. This is subject to change depending on who's working on which parts and if the team is able to meet in person on certain days. If there are blocking issues,

issues on feature branches multiple members are working on, etc., involved team members will schedule an independent meeting to hash things out.

8.2 Communication Plans

The team has decided to communicate on Discord and host meetings on Teams. We decided on using Teams specifically for Meetings because it can generate a transcript of the meetings which can be used to aid in creating the meeting minutes. For all deliverables that require write-ups, the team decided on using Google Docs due to its ease of use when working with others simultaneously. The comment feature on Google Docs will be used for delegating work and leaving questions for others, but for any pressing questions we will be using our Discord chat. We plan to look into the Milestones feature on GitHub to keep track of milestones, and also assign work through GitHub as well.

9. Team Member Roles:

Team Member	Primary Role	Secondary Role	Responsibilities
Arin Khandelwal	Front-end Developer	Machine Learning Developer	● Developing the website. ● Developing, testing, training model.
Dhruv Chand	Full-stack Developer	Machine Learning Developer	● Developing communication between front-end and back-end. ● Developing, testing, training model.
Gaurav Sharma	Group Coordinator & Machine Learning Developer.	Front-end Developer.	● Assigning tasks, running meetings. ● Developing, testing, training model. ● Developing the website.
Ha Nguyen	Full-stack Developer	DevOps & Front-end Developer	● Developing communication between front-end and back-end. ● Supporting front-end development and GitHub management.
Malko-Bani Somo	Machine Learning (Performance) Developer	Full-stack Developer	● Developing, testing, training model. ● Optimizing video and machine learning processing.

			● Developing communication between front-end and back-end.
Roshaan Quyum	Machine Learning Developer	DevOps & Full-stack Developer	● Developing, testing, training model ● Manage GitHub, enforce Git rules, set up GitHub actions ● Developing communication between front-end and back-end.
Vinoth Robinson	Front-end Developer	Machine Learning Developer	● Developing the website. ● Developing, testing, training model.

10. Workflow Plan:

10.1 Version Control

For this project, we will use GitHub for version control. The planned guidelines for contributing to the repository are as follows.

Main branch:

- Will be considered our "production" branch and must always remain in a functional state.
- Changes to the main branch may only be made via Pull Requests (PRs) and must preserve overall functionality.

Branching:

- To add to the repository, a new branch must be forked from the latest main branch.
- Changes must be made locally, and all unit tests must pass before creating a PR.
- Team members working on the same feature should have their own branches and when finished, merge them into each other before merging into main.

Pull Requests:

- Prior to submitting a PR:
  - Members must validate that their code is working correctly.
  - Members are encouraged to squash less important commits to keep a clean commit history.
  - If changes have been made to main since the branch was created, members must rebase their branch to the latest version of main and revalidate their code.
- PRs must be reviewed by at least two team members before merging.
- After approval, the author of the PR will be responsible for merging to the main branch and resolving any merge conflicts that may occur.

- After merging, the associated branch should be immediately deleted.
    - However, if a feature's requirements change, there is an increase in scope, etc., but progress has been made that constitutes a reasonable addition to main, then development can continue on the branch after a PR.

Actions:

- After each push to the main branch, GitHub Actions will validate the latest version of the model as well as all unit tests, helping flag major issues from a merge.
- If this automated check ever fails, or the reported validation of the model has experienced a serious degradation, it will be considered a P0 issue and significant resources will be allocated to restore functionality.

Issues:

- Problems in the main branch will be reported and tracked using Github Issues
- Issues must be:
    - Related to a milestone (e.g. bugs in the model should be labelled under the "Implement ML Model" milestone).
    - Labelled using tags under a rating scale from most important to least important as follows: blocking, urgent, bug, nice-to-have.
- Issues will be assigned to a team member based on their responsibilities and/or their involvement in the issue (self-assignment is allowed).
- Once an issue is resolved, the fix should follow the same workflow as any other update to main, and the associated issue can be closed.

Conventions:

- PR titles should follow the format "issue-number:category/changes-made" where "issue-number" can be omitted if the change is not associated with an issue
- Branch names should follow the hierarchical format "category/changes-made" with no white space characters. Where the category is one of the following:
    - feat: for new features or significant changes
    - fix: for bug fixes
    - chore: not directly affecting the code meanings (updating documentations, etc.)
    - test: for unit tests
    - ref: for refactoring code
- Commit messages should list what has been updated but do not need to adhere to any strict format.

10.2 Development Methodology

We will follow the Agile software development methodology, specifically Scrum, working in 1-week sprints. Each sprint will begin and end with our weekly meetings on Friday. Every team

member will be assigned a deliverable and will be expected to complete or make meaningful progress on it by the end of the sprint period. Any unfinished tasks will be carried over and prioritized into the next sprint.

10.3 Data Storage

All datasets used to train and test our model for this project will be stored in a dedicated One Drive folder tied to a team member's McMaster account. One Drive will be used over similar cloud storage services due to each McMaster student having one terabyte of storage. This will help prevent bloat in the repository while ensuring that the data still remains easily accessible. However, a small subset of the dataset will be stored on GitHub to automate validation with GitHub Actions. Outside of processing, no user data will be collected or stored. After user data is processed, it will be promptly deleted.

10.4 Compute-heavy Tasks

Compute-heavy tasks, such as training our model, will be primarily performed on McMaster's GPU clusters. By SSHing into the server, importing our datasets and training script, we will be able to efficiently run and train the model using high-performance resources. Additionally, we have access to RTX 4070 Mobile, 3070 and 3060 GPUs, which will be used for prototyping or training if the server ever becomes inoperational.

10.5 Accomplishing Requirements

In order to achieve our goal of translating ASL gestures into English text, we will use Python with computer vision and machine learning libraries outlined in section 12.2. To achieve the word-level recall and sentence-level accuracy we defined in section 5.3, we will do extensive validation with a validation dataset, consisting of approximately 10% of the original dataset. After each validation cycle, we will analyze the shortcomings of the model, allowing us to iteratively refine the model's recall and accuracy.

In order to achieve our goal of translating in real time, we will use a dynamic Vue front end that communicates with the back-end model through WebSockets, allowing for continuous low latency data exchange. To achieve the time-efficiency metric outlined in section 5.3, we will conduct extensive profiling and optimization on each step of the pipeline. In cases where Python is a bottleneck, we will consider rewriting certain sections in C++ and using PyBind to integrate the C++ code back with the rest of the codebase.

11. Proof of Concept Demonstration Plan:

The proof of concept demonstration will introduce the problem, stakeholders, and features of our application. Afterwards, we will run through a demo of a working version of the model and front

end. We will end off by talking about our next steps for improving the application. The following points summarize the main topics we will cover in each section of the presentation.

Problem:

- Approximately 70 million people around the world use sign language.
- People who use sign language primarily have difficulties with communicating with friends, family, and others.
- Existing translation solutions are often difficult to access and may not provide adequate security.

Goal:

- Develop a web application that translates a string of ASL gestures to written English in real time.
    - Provides an easy to access way that lessens the communication gap between ASL and Non-ASL users.
- Develop games and other fun activities that rely on sign language
    - Creates an environment for non-ASL users to learn sign language and engage in using it.

Stakeholders:

- Non-ASL users:
    - Would use the application to translate ASL and understand what an ASL user is communicating to them.
    - Would use the application to slightly learn some ASL through games and fun activities.
- Non-fluent ASL users:
    - Would use the application to self-study and receive feedback instantly.
    - Would use the application to engage in using ASL through the scenarios feature.
- Fluent ASL users:
    - Would use the application to translate what they are signing.
    - Would use the application to engage in using ASL through the scenarios feature.

Features:

- Planned to be completed by demonstration date:
    - A model that gives real-time translation to ASL with little delay.
    - A modern website that allows users to dynamically communicate with a backend.
- Highlighted in-progress features:
    - Using text-to-speech to verbalize the translation.
    - Syntactic models to transform output into a grammatically accurate sentence.
    - A static scenario system that engages the user in responding to situations in ASL.
    - Games to help engage users in sign language.

Demo:

- Discuss dataset and preprocessing working flows
- Showcase and describe a working prototype model.
    - Translates multiple ASL gestures into individual words.

    - Discuss current model metrics .
- Showcase front-end website, highlighting accessibility.
    - Has real-time video preview.

Next Steps:

- Further optimize model efficiency and flexibility.
- Introduce later priority features that will increase functionality.

12. Technology:

12.1 Front-end

- The backbone of front-end development will be Vite, a development environment / build tool that handles config and provides modern development features (like hot-module reloading).
    - Will specifically use Vite PWA, which handles progressive web app config.
    - We aim for a PWA to have a seamless experience developing a platform independent mobile app, all while using familiar tooling (Javascript).
    - Will use Vitest, a testing framework native to Vite.
- Will use Vue as our framework due to its simplicity.
- Will use Typescript specifically for more maintainable code.
- Will use component libraries like Volt to not need to reinvent the wheel for common components (side menus, pop-ups, in-app notifications (Toast), etc.).
- Will use Tailwind CSS / regular CSS for app styling.
- May use Mediapipe for keypoint/landmark extraction on the client side.

Will use WebSockets to handle the real-time communication between front and back end.

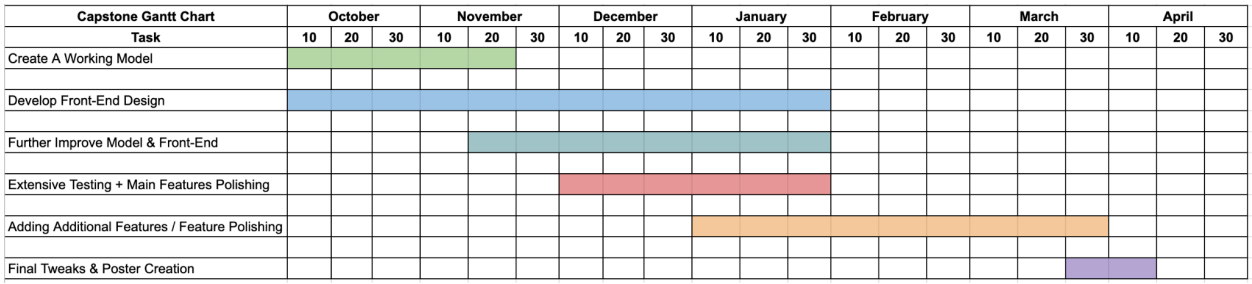
12.2 Back-end

- The backend consists of the endpoints (Websockets, maybe REST) and the model, which will be orchestrated by Python.
    - Will use ML/numeric libraries such as Numpy, OpenCV, Pytorch.
    - Will use FastAPI to handle endpoints.
    - Will use PyTest for testing.
- Bottlenecks can be detected through profiling (with perf for example), and can be optimized in C++ and exposed to Python via PyBind.
    - We may consider using opencv's C++ bindings.
    - Tools for more efficient code include SIMD intrinsics for vectorization, OpenMP for parallelization, etc.
    - Can export our Pytorch model to ONNX format for performance.

12.3 Code Environment:

- Pyenv will be used to set a default Python version, ensuring everyone is using the same version. Also, it handles setting up the virtual environment to ensure everyone uses the same versions of all required packages.
    - Setup instructions in the repository's README.md.
- Members will use their own preferred code editor (VS Code) for fixing merge conflicts.

13. Project Scheduling:

This section includes a brief overview of how we plan to structure our work for this project alongside the Gantt Chart to visualize this:

- Oct 10 - Nov 21: Prioritize creating a working model that can translate ASL gestures to words / sentences.
    - Could be a bare bones implementation.
        - Instead of full sentences, just individual words could be fine.
    - Decide what kind of model we need to use with what hyperparameters.
    - Create features from our main dataset to train on.
    - Connect it to a basic frontend before the demo date.
- Oct 10 - Jan: Work on developing the basic front end design.
- Nov 22 - Jan: Further improve the model and improve frontend design.
- Dec-Jan: Extensive testing and main features polishing.
- Jan - Mar: If on track, look into adding additional features, otherwise, focus on polishing the current model.
    - Additional features include some of the following based on our initial proposal:
        - Image responder (flash an image on screen, then sign it within a time limit, like geoguesser).
        - Translation questions (i.e, what is this sentence in ASL).
        - A scenario system that engages the user in responding to day-to-day situations in ASL.
        - Employing syntactic models to convert the string of words into a grammatically accurate sentence.
        - Using facial expressions to denote tone and grammar.
        - In addition to tone and grammar, using text-to-speech software to accurately convey a sentence with emotion.
- Late Mar - Apr: Final tweaks, and poster creation.